Sistemas Inteligentes focados em Redes Neurais de Pulso e Identificação de Locutores via voz e Eletro-encefalograma.

André Furlan - ensismoebius@gmail.com

https://github.com/ensismoebius

2026

Fundamentos de redes neurais

Como uma rede neural é construída

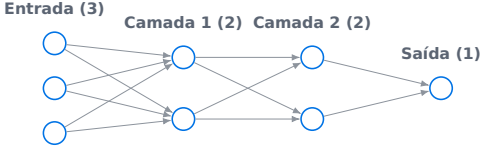
O problema: aprender a transformação em vez de programá-la à mão

Dado um sinal de entrada (um pixel, uma amostra de áudio, uma janela de EEG) e a resposta certa que se quer para ele, como chegar da entrada à resposta sem escrever a regra manualmente? A rede neural é essa transformação — com parâmetros que o próprio treino ajusta, em vez de uma fórmula fixada por quem programou.

Uma rede neural é organizada em **camadas**:
entrada $\rightarrow$ uma ou mais camadas ocultas $\rightarrow$ saída.

Cada neurônio recebe as saídas da camada anterior vezes seus **pesos** (parte linear), soma tudo, e aplica uma **ativação** (parte não linear).

Sem a não linearidade, empilhar camadas equivaleria a uma única camada linear — é a ativação que dá à rede o poder de aproximar funções complexas.



O que é um peso, e como ele é calculado

Um **peso** é um número real associado a uma conexão entre dois neurônios — é o parâmetro que a rede efetivamente **aprende**. Os dados de entrada não mudam; os pesos, sim.

Dentro de um neurônio, os pesos entram na **combinação linear**: cada entrada é multiplicada pelo seu peso, e os produtos são somados.

$$z = \sum_i w_i x_i \qquad y = \sigma(z)$$

Exemplo numérico real com o primeiro neurônio da rede do slide anterior:

Entrada $x = [1{,}0,\ 0{,}5,\ -0{,}5]$
Pesos $w = [0{,}4,\ -0{,}3,\ 0{,}6]$.

$$z = 0{,}4 \cdot 1{,}0 + (-0{,}3) \cdot 0{,}5 + 0{,}6 \cdot (-0{,}5) = -0{,}05 \quad \Longrightarrow \quad y = \sigma(-0{,}05) \approx 0{,}4875$$

Cada um dos outros neurônios repete exatamente esse cálculo, com sua própria linha de pesos. Antes do treino, os pesos partem de algum valor inicial (aqui, fixos); o que o treino faz é ajustá-los, um pouco a cada passo, para que a saída da rede se aproxime do que se quer.

Como uma rede neural é treinada

1. Avanço (Forward): calcula a saída y a partir da entrada e dos pesos atuais.

2. Perda (loss): compara y com o alvo, num único número que cresce com o erro.

3. Retropropagação (backward): a regra da cadeia calcula $\partial L/\partial w$ — quanto a perda mudaria se cada peso mudasse um pouco.

4. Atualização: cada peso anda um passo no sentido **contrário** ao gradiente.

$$w \leftarrow w - \eta \cdot \frac{\partial L}{\partial w}$$

Exemplo numérico real (neurônio com entrada: $x = 2{,}0$, alvo: $= 0{,}9$, $w_0 = -1{,}0$, $\eta = 3{,}0$):

$$z = w_0 x = -2{,}0 \Rightarrow$$
$$y = \sigma(z) \approx 0{,}1192 \Rightarrow$$
$$L = \tfrac{1}{2}(y - \text{alvo})^2 \approx 0{,}3048$$

$$\frac{\partial L}{\partial w} = \underbrace{(y - \text{alvo})}_{\partial L/\partial y} \cdot \underbrace{y(1-y)}_{\sigma'(z)} \cdot \underbrace{x}_{\partial z/\partial w} = (-0{,}7808) \cdot 0{,}1050 \cdot 2{,}0 \approx -0{,}1640$$

$$w_1 = -1{,}0 - 3{,}0 \cdot (-0{,}1640) \approx -0{,}508$$

Repetindo os quatro passos, y converge para o alvo. É exatamente esse ciclo que se questiona: E se os pesos só valerem $\{-1, 0, 1\}$, ou se a ativação virar um pulso discreto?

Demonstração ao vivo

Backpropagation, Rede 3-2-2-1

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh backprop.mlp
```

Demonstração ao vivo

Backprop → Forward e backward clássicos

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh backprop.classic
```

A camada inteira é uma multiplicação de matrizes

Os slides anteriores calcularam **um neurônio de cada vez**. Mas a camada toda é **uma única operação**: empilhe os pesos de cada neurônio como **linhas** de uma matriz W, e o produto escalar de cada neurônio passa a ser uma linha do produto Wx.

A regra do mapeamento é esta:
linha = neurônio de *destino*;
coluna = neurônio de entrada.

$$z = Wx \qquad y = \sigma(z)$$

Exemplo numérico real, com a rede da demonstração ($2 \to 2 \to 1$):

$$\underbrace{\begin{bmatrix} 0{,}8 & -0{,}4 \\ -0{,}5 & 0{,}9 \end{bmatrix}}_{W_1} \underbrace{\begin{bmatrix} 0{,}9 \\ 0{,}4 \end{bmatrix}}_{x} = \underbrace{\begin{bmatrix} 0{,}56 \\ -0{,}09 \end{bmatrix}}_{z} \xrightarrow{\sigma} \underbrace{\begin{bmatrix} 0{,}6365 \\ 0{,}4775 \end{bmatrix}}_{y}$$

A primeira linha do resultado é $0{,}8 \cdot 0{,}9 + (-0{,}4) \cdot 0{,}4 = 0{,}56$: o mesmo $\sum_i w_i x_i$ do slide anterior, agora lido como “linha vezes vetor” (Conhecido como valor **z**). Já a ativação é um passo **separado**: σ age em cada célula isoladamente, sem misturar linhas — ao contrário da multiplicação, que existe justamente para combiná-las.

O backward é a mesma matriz, transposta

No forward, W_2 levou 2 valores (os ocultos H_1, H_2) para 1 (a saída O). O gradiente faz o caminho **inverso**: de 1 volta para 2. A matriz que faz isso é a **mesma**, só que transposta: o que era **linha** vira **coluna**. **O erro volta pelos próprios pesos do forward**, não por pesos novos.

$$\frac{\partial L}{\partial x} = W^\top \frac{\partial L}{\partial z} \qquad \frac{\partial L}{\partial W} = \frac{\partial L}{\partial z} \otimes \text{entrada}$$

Continuando a rede do slide anterior ($H_1 \approx 0{,}6365$, $H_2 \approx 0{,}4775$) até a saída O, com $W_2 = \begin{bmatrix} 1{,}2 & -0{,}9 \end{bmatrix}$ (1 linha: 2 entradas $\to$ 1 saída) e alvo $= 0{,}9$:

$$z_O = W_2 \cdot H = 1{,}2 \cdot 0{,}6365 + (-0{,}9) \cdot 0{,}4775 \approx 0{,}3340 \Rightarrow$$

$$y_O = \sigma(z_O) \approx 0{,}5827 \Rightarrow$$

$$L = \tfrac{1}{2}(y_O - \text{alvo})^2 \approx 0{,}0503 \;\Rightarrow\; \frac{\partial L}{\partial y_O} = y_O - \text{alvo} \approx -0{,}3173$$

A cadeia completa: de $\partial L/\partial y_O$ até o peso w_{11}

Passo 1 — atravessa a sigmoide de O:

$$\frac{\partial L}{\partial z_O} = \frac{\partial L}{\partial y_O} \cdot \sigma'(z_O) \approx (-0{,}3173) \cdot 0{,}2432 \approx -0{,}0771$$

Passo 2 — volta por $W_2^\top$ (a linha vira coluna: 1 escalar $\rightarrow$ 2 valores, um por neurônio oculto):

$$\underbrace{\begin{bmatrix} 1{,}2 \\ -0{,}9 \end{bmatrix}}_{w_2^\top} \cdot (-0{,}0771) = \begin{bmatrix} -0{,}0926 \\ 0{,}0694 \end{bmatrix} = \begin{bmatrix} \partial L/\partial H_1 \\ \partial L/\partial H_2 \end{bmatrix}$$

Passo 3 — atravessa a sigmoide de H_1 e chega no peso w_{11} (de x_1 para H_1):

$$\frac{\partial L}{\partial z_{H_1}} = \frac{\partial L}{\partial H_1} \cdot \sigma'(z_{H_1}) \approx (-0{,}0926) \cdot 0{,}2314 \approx -0{,}0214$$

$$\frac{\partial L}{\partial w_{11}} = \frac{\partial L}{\partial z_{H_1}} \cdot x_1 \approx (-0{,}0214) \cdot 0{,}9 \approx -0{,}01928$$

Mesmo resultado de antes — agora com cada multiplicação da cadeia visível, não só o produto final.

Demonstração ao vivo

Backprop → A rede como matrizes

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh backprop.matrix
```

Demonstração ao vivo

Backprop → A regra da cadeia como máquina de Rube Goldberg

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh backprop.rube_goldberg
```

Clique na tela para empurrar a bolinha por cada engenhoca – física real: ela rola pelas rampas, tomba a alavanca do peso w_2 (negativo) e gira a roldana, trocando de cor quando o sinal inverte.

Demonstração ao vivo

Backprop → Camadas e a regra da cadeia

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh backprop.chain
```

Percorra os passos de $\partial L/\partial a_2$ até $\partial L/\partial b_1$: cada carta é a derivada local de um bloco, na coluna do bloco. Feche com os cinco fatores em fila.

Redes de pulso

Uma RNA convencional ativa **todos** os neurônios em **toda** passada — o custo energético não depende do conteúdo do dado, só do tamanho da rede.

O neurônio biológico só dispara quando o potencial acumulado ultrapassa um limiar; fica quieto o resto do tempo.

Maass (1997) chamou as RNPs de *”modelos de redes neurais **terceira geração**”* (MAASS, 1997) — depois do perceptron (1ª geração) e das redes com ativação contínua/sigmoide (2ª geração).

Uma RNP **não** é uma simulação um-para-um de neurônios biológicos. Ela aproxima certas capacidades computacionais de propriedades biológicas específicas — não o neurônio inteiro.

Modelos mais fiéis à biologia existem (Hodgkin–Huxley (GERSTNER et al., 2014), dendritos não-lineares), mas são caros demais para treinar em escala. As RNPs usadas em aprendizado de máquina adotam modelos simplificados, como o **LIF** (próximo slide).

RNA vs. RNP: o par que se confunde

Rede Neural Artificial (RNA)

Ativação: valor **contínuo** por neurônio, por camada.
Processamento **síncrono**: uma passada completa por amostra.
Sem dimensão temporal intrínseca (salvo recorrente).
Custo computacional fixo, independente da atividade.

Rede Neural de Pulso (RNP)

Ativação: **pulso** (0 ou 1), esparso no tempo.
Processamento **orientado a evento**: só computa quando há pulso.
Dinâmica temporal **intrínseca**: potencial de membrana acumula.
Informação em **taxa** e/ou **tempo** de disparo (KASABOV, 2019).

Consequência prática

Uma RNP processa naturalmente **fluxos** (áudio, EEG, câmera de eventos) — um dado estático exige “esticá-lo” por vários passos de tempo, custo que a RNA não tem.

Demonstração ao vivo

SNN → Normalização: por característica x por janela

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.normalization
```

Áudio padroniza cada característica com média/desvio ajustados uma vez no treino (CMVN); EEG recalcula a cada janela, e uma deriva de +50 μV some. Só o caminho ajustado pode vazar dados de teste – em silêncio.

Demonstração ao vivo

SNN → Sinal e spikes

▶ **Abrir no software**

ou, no terminal (a partir desta pasta):

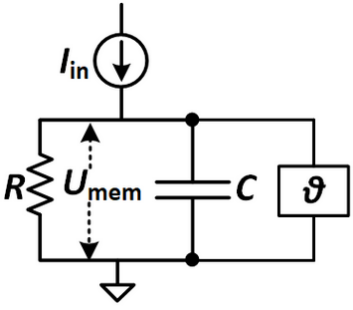
```
./abrir-demo.sh snn.spikes
```

O LIF se comporta como um circuito Resistor-Capacitor (RC).

R: resistência de vazamento; C: capacitância; V_{mem}: potencial acumulado; ϑ: interruptor que descarrega o capacitor (dispara um pulso) ao atingir o limiar.

Governado por (ESHRAGHIAN et al., 2023):

$\tau \frac{dV_{mem}}{dt} = R\, I_{in}(t) - V_{mem}(t)$, com $\tau = R \cdot C$.



Fonte: (ESHRAGHIAN et al., 2023)

Sem corrente ($I_{in} = 0$), a equação do slide anterior vira $\tau\, dV_{mem}/dt = -V_{mem}$. Separando as variáveis e integrando os dois lados:

$$\frac{dV_{mem}}{V_{mem}} = -\frac{dt}{\tau} \;\Rightarrow\; \int \frac{dV_{mem}}{V_{mem}} = \int -\frac{dt}{\tau} \;\Rightarrow\; \ln V_{mem} = -\frac{t}{\tau} + k$$
$$\Rightarrow\; V_{mem}(t) = e^{k} \cdot e^{-t/\tau} \;\Rightarrow\; V_{mem}(t) = V_{mem}(0) \cdot e^{-t/\tau}$$

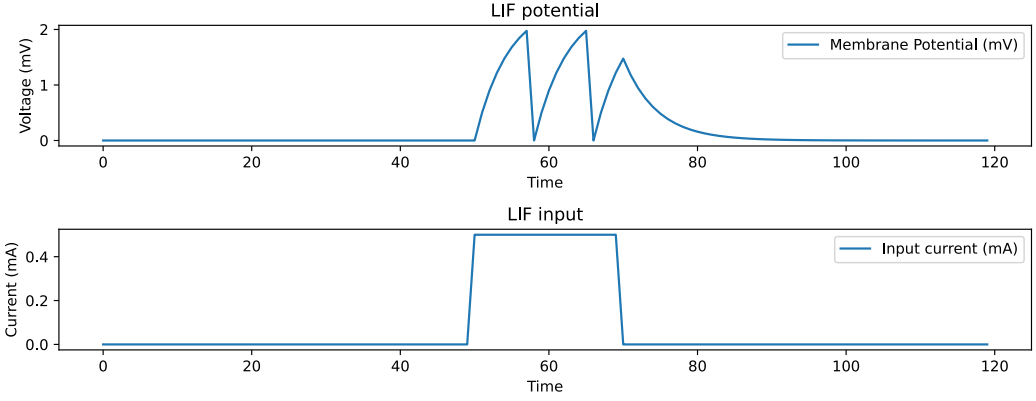
(pois em $t = 0$: $e^{k} = V_{mem}(0)$)
Com $R = 5\,\Omega$, $C = 1\,F$ (logo $\tau = R \cdot C = 5$) e potencial inicial $V_{mem}(0) = 1$:

$$V_{mem}(1) = 1 \cdot e^{-1/5} \approx 0{,}8187$$

O potencial **vaza** exponencialmente na ausência de estímulo — daí o nome *leaky*. Quando um estímulo chega, o potencial sobe (mesma matemática, sinal oposto) até cruzar um limiar V_{th}, disparando um pulso e reiniciando.

O neurônio *Leaky Integrate-and-Fire* (LIF)

Comportamento completo simulado



Corrente de 0,5 mA injetada entre os passos 51 e 70. Fonte: elaborado pelo autor.

Demonstração ao vivo

SNN → LIF

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.lif
```

Os parâmetros padrão (tau=5, R=5, V_th=1) já reproduzem o exemplo numérico anterior; aperte Play para o comportamento completo.

time_steps (T) e Δt: o que são?

Discretização do tempo contínuo em redes neurais de pulso

Redes biológicas e sinais físicos operam em **tempo contínuo**. Ao simular uma SNN em computadores digitais, o tempo precisa ser fatiado em intervalos discretos — como os quadros de um filme.

	time_steps (T)	**Passo de tempo (Δt)**
Conceito	Quantidade de iterações/quadros	Duração física de cada quadro
Responde a	Quantos passos calcular?	Quanto tempo passa por passo?
Exemplo	$T = 16$ passos	$\Delta t = 1{,}0$ ms por passo

Janela temporal simulada total: $t_{\text{total}} = T \cdot \Delta t$.
Com $T = 16$ passos e $\Delta t = 1$ ms, simula-se 16 ms. Se $\Delta t = 0{,}1$ ms, os mesmos 16 passos cobrem apenas 1,6 ms de física real.
T dita o **esforço computacional** (número de laços temporais); Δt dita a **escala temporal física** da dinâmica neuronal (GERSTNER et al., 2014; ESHRAGHIAN et al., 2023).

time_steps (T) e Δt: qual a sua importância?

Impacto na memória do neurônio LIF e na estrutura dos tensores

Memória e decaimento do potencial (β): No neurônio LIF, a fração do potencial que sobrevive entre passos sucessivos é $\beta = \exp(-\Delta t/\tau)$, onde $\tau = RC$ é a constante de tempo da membrana (resistência $R\times$ capacitância C). Com $\tau = 5\,\text{ms}$:

Se $\Delta t = 1\,\text{ms}$: $\beta = e^{-0{,}2} \approx 0{,}82 \implies$ a membrana retém 82% da carga (memória temporal ativa).

Se $\Delta t = 16\,\text{ms}$: $\beta = e^{-3{,}2} \approx 0{,}04 \implies$ o potencial vaza quase todo em um único passo (perda de memória).

Custo computacional e BPTT: Mais passos (T alto) capturam dinâmicas ricas, mas elevam tempo e memória de treino linearmente ($O(T)$).

Armadilha na ordem dos tensores: Em tensores com dimensões Tempo (T), Lote (B, *batch size*) e Características (F, *features*): (T, B, F) indexa a linha por $t \cdot B + b$, enquanto (B, T, F) indexa por $b \cdot T + t$ (onde $t \in [0, T-1]$ e $b \in [0, B-1]$). Se o código espera (T, B, F) mas recebe (B, T, F), a linha $b_0 t_1$ (segundo passo da amostra 0) é lida como "amostra 1 no passo t=0": o LIF encadeia a memória de **duas amostras distintas** como se fosse uma mesma linha do tempo, gerando uma falha silenciosa no aprendizado.

Referências: (GERSTNER et al., 2014; ESHRAGHIAN et al., 2023).

Demonstração ao vivo

SNN → time_steps x delta_t

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.timesteps
```

Quantos quadros x quanto dura um quadro – e por que a ordem das linhas no tensor (T*B,F) importa.

Codificação Poisson: o que é?

Mapeando intensidade contínua em probabilidade de disparo de pulsos

Sinais reais (pixels, áudio, medidas analógicas) são contínuos ($x \in [0,1]$). Neurônios de pulso comunicam-se via eventos binários discretos ($s_t \in \{0,1\}$). A **codificação Poisson** (por taxa estocástica) converte a intensidade na **probabilidade de disparo por passo**:

$$p_t = x \cdot r_{\max}, \qquad s_t \sim \text{Bernoulli}(p_t)$$

onde $x \in [0,1]$ é a entrada, $r_{\max} \in (0,1]$ a taxa máxima, p_t a probabilidade no passo t e $s_t \in \{0,1\}$ o disparo (1 = pulso, 0 = silêncio).

Entrada (x)	**Probabilidade (p)**	**Média $\mathbb{E}[N]$ ($T = 10$)**
Pixel escuro ($x = 0{,}1$)	$p = 0{,}09$ (9%)	$\mathbb{E}[N] = 10 \times 0{,}09 = 0{,}9$ pulso
Pixel brilhante ($x = 0{,}6$)	$p = 0{,}54$ (54%)	$\mathbb{E}[N] = 10 \times 0{,}54 = 5{,}4$ pulsos
Pixel saturado ($x = 1{,}0$)	$p = 0{,}90$ (90%)	$\mathbb{E}[N] = 10 \times 0{,}90 = 9{,}0$ pulsos

$N = \sum_{t=1}^{T} s_t$ é a contagem total de pulsos acumulada nos T passos. A informação reside na **taxa média de disparos** ao longo da janela simulada.

Codificação Poisson: qual a sua importância e limitações?

Ponte para bases convencionais e os custos da estocasticidade

Importância fundamental:

Inspiração neurobiológica direta (codificação por taxa média no córtex).

Permite alimentar SNNs com qualquer conjunto de dados estático clássico (MNIST, CIFAR) sem sensores neuromórficos nativos (como DVS) (DIEHL et al., 2015).

Limitações e o custo da variância:

Variância de amostragem: duas repetições do mesmo estímulo geram sequências diferentes de pulsos.

Custo de latência e energia: estimar a taxa pela frequência $\hat{p} = N/T$ com desvio padrão $\sigma = \sqrt{p(1-p)/T}$ exige T grande. Para o pixel brilhante anterior ($p = 0{,}54$): em $T = 10$, $\sigma \approx 0{,}158$ (29% de erro relativo); em $T = 50$, $\sigma \approx 0{,}070$ (13%) — $5\times$ mais passos (mais pulsos, mais tempo, mais energia) para menos que a metade do erro.

Com poucos passos (T curto), essa dispersão atua como um ruído intrínseco.

O que observar na demonstração a seguir

O mesmo sinal alimenta cada passo. A intensidade define a probabilidade de pulso, produzindo um padrão pontilhado no tempo.

Referências: (DIEHL et al., 2015; ESHRAGHIAN et al., 2023).

Demonstração ao vivo

SNN → Codificação Poisson

▶ **Abrir no software**

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.poisson
```

Demonstração ao vivo

SNN → Codificação Poisson (imagem)

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.poisson_image
```

Ruído estrutural da codificação: o que é?

O piso mínimo de perda de informação antes de qualquer processamento

Ao converter um valor contínuo $x \in [0, 1]$ em trens de pulso e depois tentar reconstruí-lo, ocorre uma perda de precisão inevitável. O **ruído estrutural** é o erro médio mínimo (o "piso de ruído") imposto pela própria matemática da codificação.

Esse piso independe do aprendizado: tomando x **uniformemente distribuído** em $[0, 1]$ (notação $x \sim \mathcal{U}[0, 1]$: todo valor entre 0 e 1 igualmente provável, só para não favorecer nenhuma entrada específica na conta) e $T = 16$ passos, o **erro RMS** — raiz do erro quadrático médio, $\sqrt{\mathbb{E}[(x - \hat{x})^2]}$, entre o valor original x e o valor $\hat{x}$ reconstruído a partir dos pulsos — de reconstrução teórico é:

Codificação	**Mecanismo de representação**	**Piso RMS ($T = 16$)**
Direta	Corrente sem quantização prévia	**0**
Latência (TTFS)	Primeiro pulso em $t \in \{0, \ldots, 15\}$	$\approx$ **0,019**
Poisson (Taxa)	Contagem de disparos no tempo	$\approx$ **0,102**

Na **latência**, o erro vem da quantização em 16 níveis discretos ($\Delta x = 1/15$). Em **Poisson**, o erro decorre da variância intrínseca dos sorteios aleatórios.

De onde vêm os números da tabela: TTFS e o cálculo do piso RMS

Nada na tabela anterior caiu do céu

TTFS (*Time To First Spike*, "tempo até o primeiro pulso"): cada neurônio dispara **um único pulso**, no passo t que codifica x — quanto **maior** x, mais **cedo** o pulso. Com $T = 16$ passos ($t \in \{0, \dots, 15\}$):

$$t = \mathrm{round}\big((1 - x) \cdot (T - 1)\big) \qquad \Longrightarrow \qquad \hat{x} = 1 - \frac{t}{T - 1}$$

Exemplo:
$x = 0{,}63 \Rightarrow t = \mathrm{round}(0{,}37 \times 15) = \mathrm{round}(5{,}55) = 6 \Rightarrow \hat{x} = 1 - 6/15 = 0{,}60$ — erro de 0,03, dentro de meio passo ($\Delta x/2 \approx 0{,}033$). É exatamente esse arredondamento, repetido para todo $x \in [0, 1]$, que produz o piso $\approx 0{,}019$ da tabela:

$$\mathrm{RMS}_{\mathrm{lat}} = \frac{\Delta x}{\sqrt{12}} = \frac{1/15}{\sqrt{12}} \approx 0{,}019$$

(um erro de arredondamento uniforme em $[-\Delta x/2, \Delta x/2]$ tem variância $\Delta x^2/12$ — resultado clássico de quantização)
Já em **Poisson**, $\hat{x}$ é a taxa estimada N/T pulsos; sua variância é $p(1-p)/T$. Integrando sobre $x \sim \mathcal{U}[0, 1]$ (logo $p = x$): $\mathbb{E}[p(1-p)] = \frac{1}{2} - \frac{1}{3} = \frac{1}{6}$, o que dá o piso $\approx 0{,}102$ da tabela:

$$\mathrm{RMS}_{\mathrm{Poisson}} = \sqrt{\frac{\mathbb{E}[p(1-p)]}{T}} = \sqrt{\frac{1/6}{16}} \approx 0{,}102$$

Ruído estrutural da codificação: qual a sua importância?

Evitando conclusões errôneas ao comparar modelos e arquiteturas

A armadilha metodológica em comparações de modelos:

Se uma rede com codificação direta supera uma rede com Poisson, a tentação é atribuir o mérito à arquitetura ou aos pesos.

Porém: a rede Poisson começou com um piso de ruído cinco vezes superior na própria entrada! A diferença observada pode ser fruto da representação, e não da rede.

Decisões de projeto e compromissos:

Codificação Direta: sem erro na entrada, mas a primeira camada perde a computação puramente orientada a eventos.

Latência (TTFS): piso de erro baixo com alta eficiência (1 pulso por neurônio).

Poisson: piso maior para T curto, porém tolerante a ruído e falhas pontuais.

Referências: (ESHRAGHIAN et al., 2023; LIU; LEUNG; DRAGOTTI, 2023).

Demonstração ao vivo

SNN → Ruído estrutural da codificação

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.encoding_noise
```

”Direta ¿ latência ¿ Poisson”pode ser só o piso de ruído estrutural de cada codificação, não uma diferença real de informação.

Perda incompatível com a codificação: o que é?

O descompasso entre onde a informação está e o que a perda avalia

Em SNNs, a saída é uma **sequência de pulsos no tempo**. A função de perda deve mensurar a grandeza exata onde a informação foi codificada.
Um **descompasso de codificação** ocorre quando a métrica ignora a dimensão temporal do código:

	Alvo desejado	**Previsão da rede**
Instante do pulso (t)	$t_{\text{alvo}} = 4$	$t_{\text{pred}} = 12$
Contagem de pulsos (N)	$N_{\text{alvo}} = 1$	$N_{\text{pred}} = 1$
Perda de contagem (L_{taxa})	$(N_{\text{pred}} - N_{\text{alvo}})^2 = (1-1)^2 = \mathbf{0} \implies$ Erro zero (ilusão)	
Perda temporal (L_{tempo})	$(t_{\text{pred}} - t_{\text{alvo}})^2 = (12-4)^2 = \mathbf{64}$ (8 passos atrasado)	

A perda por contagem calcula gradiente zero e não corrige nada, embora o pulso tenha chegado completamente atrasado!

Perda incompatível com a codificação: qual a sua importância?

Duas falhas silenciosas que sabotam o treinamento de SNNs

Falha Silenciosa 1: Otimização cega por incompatibilidade:

As curvas de perda caem a zero e o otimizador reporta convergência, mas o modelo falha na tarefa porque a resposta temporal nunca foi ajustada.

Falha Silenciosa 2: Gradientes mortos na perda temporal (*SpikeTimeLoss*):

Para códigos temporais, mede-se a perda quadrática $L = (t_{\text{pred}} - t_{\text{alvo}})^2$.

Se um neurônio **nunca dispara** em nenhum dos T passos, adota-se a convenção de penalidade máxima $t_{\text{pred}} = T$ (o fim da janela temporal).

A perda reportada é alta, mas como o neurônio nunca cruzou o limiar, a derivada do tempo de disparo não fornece gradiente para erguer o potencial!

O neurônio entra em silêncio irreversível durante o restante do treino.

Diretriz: Alinhar estritamente a perda ao código (taxa $\leftrightarrow$ contagem; latência $\leftrightarrow$ instante) e monitorar neurônios inativos.

Referências: (ESHRAGHIAN et al., 2023; LIU; LEUNG; DRAGOTTI, 2023).

Demonstração ao vivo

SNN → Perda incompatível com a codificação

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.encoding_loss_mismatch
```

Perda de contagem sobre latência: o treino reporta perda zero sem corrigir nada. Perda certa (SpikeTimeLoss): uma unidade que nunca dispara fica sem gradiente. Duas falhas silenciosas – e a validação que as torna barulhentas.

Como treinar uma função que não tem gradiente?

Quatro famílias de resposta, historicamente

A função de disparo do LIF se aproxima de uma **função em degrau** (Heaviside): dispara ou não dispara, sem meio-termo. Sua derivada é zero em quase todo ponto — a descida de gradiente padrão não tem o que propagar (KASABOV, 2019).
Quatro famílias de resposta, historicamente:

STDP (*Spike-Timing-Dependent Plasticity*, "plasticidade dependente do tempo entre pulsos"): regra local — se o pré-sináptico dispara **antes** do pós-sináptico ($\Delta t = t_{\text{pós}} - t_{\text{pré}} > 0$), a conexão se fortalece; se depois ($\Delta t < 0$), se enfraquece. Biologicamente plausível, não supervisionada.

Gradiente substituto + BPTT (*Backpropagation Through Time* — detalhada nos próximos slides): usa a função em degrau real no *forward*, e uma aproximação suave e diferenciável só no *backward*.

Algoritmos evolutivos: seleção dos hiperparâmetros mais aptos ao longo de gerações.

Reservatório / computação dinâmica: treina só a camada de saída, mantendo pesos recorrentes fixos (*echo state networks*, *liquid state machines*).

Como treinar uma função que não tem gradiente?

Alternativa: converter em vez de treinar

Treina uma **RNA convencional**, com técnicas padrão de otimização.

Converte os pesos treinados para uma RNP, balanceando pesos e limiares de disparo (DIEHL et al., 2015; RUECKAUER et al., 2017).

Evita o problema do gradiente por completo — mas herda limitações: geralmente precisa de **mais passos de tempo** para aproximar a acurácia da RNA original, e nem toda arquitetura converte bem.

Demonstração ao vivo

SNN → Surrogate gradient

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.surrogate
```

Percorra os checkpoints ”A derivada real”até ”O gradiente substituto”.

Retropropagação através do tempo (BPTT): o que é?

Estendendo a descida de gradiente para redes com dinâmica e memória

Em redes convencionais *feedforward*, a informação viaja apenas de camada em camada ($l \to l+1$), sem memória de estados passados.
Em uma SNN, cada neurônio LIF acumula potencial de membrana u_t (equivalente discreto do potencial $V_{mem}(t)$) que evolui ao longo de T passos temporais:

A resposta no passo atual t depende diretamente do que aconteceu nos passos anteriores.

Se a rede erra em $t = T$, o erro pode ter sido causado por um pulso disparado lá atrás em $t = 1$.

A **Retropropagação através do tempo** (*Backpropagation Through Time* — BPTT) calcula o gradiente do erro retrocedendo **entre as camadas e através dos passos de tempo**.

Por que ela é necessária?

Sem o BPTT, a rede não aprende atribuição de crédito temporal (*temporal credit assignment*): treinar cada passo isoladamente apagaria a memória da dinâmica de membrana.

BPTT: a intuição temporal

Um neurônio vê um filme, não uma foto isolada

Exemplo com neurônio LIF: potencial $u_t = \beta\, u_{t-1} + x_t$ (onde $\beta = 0{,}5$ é o decaimento e x_t a entrada no instante t), limiar $V_{th} = 1{,}0$ e reset para 0 após disparo:

Passo t	**Entrada** x_t	**Dinâmica** u_t	$u_t \geq 1{,}0$**?**	**Pulso?**
1	0,8	$0{,}5(0) + 0{,}8 = 0{,}8$	Não	Não
2	0,0	$0{,}5(0{,}8) + 0 = 0{,}4$	Não	Não (decaimento retido)
3	0,8	$0{,}5(0{,}4) + 0{,}8 = 1{,}0$	**Sim**	**Sim** (dispara e reseta)

A dependência temporal na prática

O pulso em $t = 3$ só ocorreu porque o estímulo de $t = 1$ ainda estava retido na memória da membrana no passo 2! Sem x_1, u_3 seria apenas 0,8 (sem disparo).

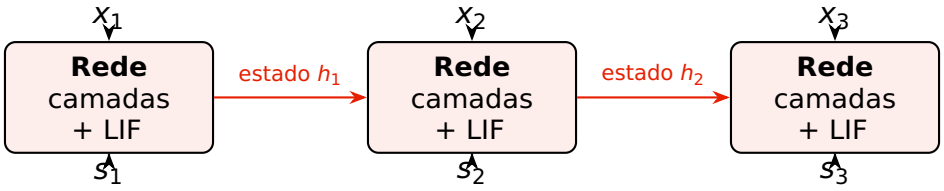
> Para ajustar os pesos corretamente, o gradiente do erro em $t = 3$ precisa viajar no tempo até $t = 1$.

BPTT: abrir a rede ao longo do tempo (*Unrolling*)

Transformando recorrência temporal em camadas equivalentes

Para calcular o gradiente no tempo, a rede é ”desenrolada”conceitualmente em T passos temporais:

Mesma rede, pesos compartilhados W, desenrolada em 3 instantes



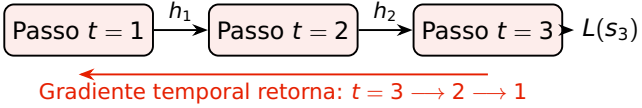
Cada bloco executa um passo temporal: x_t é a entrada, $s_t \in \{0, 1\}$ é o pulso emitido e h_t é o estado interno de membrana entregue ao passo seguinte.

Pesos compartilhados: Todas as réplicas temporais utilizam rigorosamente os mesmos pesos W.

BPTT: o caminho retrógrado do erro

Propagação do gradiente no espaço e no tempo

No *forward*, o tempo avança de $t = 1$ até T. No *backward*, a regra da cadeia percorre o grafo no sentido inverso:



Acumulação nos pesos: Como W atua em todo t, a atualização total $\frac{\partial L}{\partial W}$ soma os gradientes locais $\frac{\partial L}{\partial W^{(t)}}$ de cada instante:

$$\frac{\partial L}{\partial W} = \sum_{t=1}^{T} \frac{\partial L}{\partial W^{(t)}}$$

A ponte com o Gradiente Substituto: Como a derivada do degrau de disparo é zero, o BPTT usa o **gradiente substituto** para fornecer uma derivada suave que permita ao erro atravessar o pulso (WU et al., 2018; ESHRAGHIAN et al., 2023).

BPTT: o mesmo exemplo, agora com números

$\partial L/\partial W$ passo a passo, continuando o exemplo da intuição temporal

Retomando o exemplo anterior, com peso de entrada W: $u_t = \beta\, u_{t-1} + W x_t$, com $\beta = 0{,}5$, $W = 1$ e vetor de entradas $x = (0{,}8;\ 0;\ 0{,}8)$ — mesma dinâmica: $u_1 = 0{,}8$, $u_2 = 0{,}4$, $u_3 = 1{,}0$. Com perda quadrática $L = \frac{1}{2}(u_3 - u_{\text{alvo}})^2$ e alvo $u_{\text{alvo}} = 0{,}9$: o erro final é $\partial L/\partial u_3 = u_3 - u_{\text{alvo}} = 0{,}1$.

Passo t	$\partial u_3/\partial u_t = \beta^{3-t}$	x_t	**Contribuição** $\frac{\partial L}{\partial u_3} \cdot \frac{\partial u_3}{\partial u_t} \cdot x_t$
1	$\beta^2 = 0{,}25$	0,8	$0{,}1 \times 0{,}25 \times 0{,}8 = 0{,}02$
2	$\beta^1 = 0{,}50$	0	$0{,}1 \times 0{,}50 \times 0 = 0$
3	$\beta^0 = 1{,}00$	0,8	$0{,}1 \times 1{,}00 \times 0{,}8 = 0{,}08$

$$\frac{\partial L}{\partial W} = \sum_{t=1}^{3} \frac{\partial L}{\partial u_3} \cdot \frac{\partial u_3}{\partial u_t} \cdot x_t = 0{,}02 + 0 + 0{,}08 = 0{,}10$$

O termo t=1 **ainda contribui** (0,02) dois passos antes do erro: é essa soma que faz o aprendizado ”olhar para trás”no tempo. Já t=2 contribui **zero**, não pela distância, mas porque $x_2 = 0$: sem entrada, não há informação sobre W.

Retropropagação através do tempo (BPTT): qual a sua importância?

Equivalência com Deep Learning moderno e desafios de memória

Importância científica e prática:

Unificou o treino de SNNs com a infraestrutura de diferenciação automática (PyTorch, JAX).

Superou regras heurísticas locais, permitindo aprendizado supervisionado de ponta a ponta.

	Rede feedforward	**SNN + BPTT**
Caminho do erro	Apenas entre camadas	Entre camadas **e no tempo**
Estados retidos	Ativações estáticas	Histórico u_t em todo $t \in [1, T]$
Memória na GPU	O(profundidade)	O(profundidade $\cdot\, T$)

Desafio prático: Janelas longas elevam a memória. Solução: BPTT Truncado (TBPTT) (ESHRAGHIAN et al., 2023).

Referências: (WU et al., 2018; ESHRAGHIAN et al., 2023).

Normalização dependente de limiar (tdBN): o que é?

Adequando a normalização por lote à física dos neurônios de pulso

Em redes clássicas, a **Batch Normalization (BN)** padroniza saídas com média 0 e variância unitária ($\sigma = 1$), acelerando a convergência.

O dilema em SNNs: Neurônios LIF disparam só quando o potencial atinge o limiar V_{th}.

Com BN clássica ($\sigma = 1$), se $V_{th} = 2{,}0$, quase toda a distribuição normalizada fica abaixo de $V_{th} \implies$ **a camada inteira silencia (zero disparos)!**

Se V_{th} for muito baixo, a camada inteira satura em rajadas descontroladas.

A **tdBN** (*Threshold-dependent Batch Normalization*) escala a corrente proporcionalmente ao próprio limiar V_{th}:

$$Y = \alpha V_{th} \cdot \frac{X - \mu}{\sqrt{\sigma^2 + \epsilon}} + \beta$$

Onde: X é a pré-ativação (corrente pré-sináptica) antes de normalizar; μ e σ^2 são a média e variância do lote (*batch*); $\epsilon > 0$ evita divisão por zero; V_{th} é o limiar de disparo; α e β são os parâmetros aprendidos de escala e deslocamento (*bias*) da normalização (distintos do β de decaimento do LIF).

tdBN: um exemplo numérico

De onde vêm os números da tabela a seguir

Um lote de 4 valores pré-ativação do mesmo canal, $X = \{-3, -1, 1, 3\}$, com escala $\alpha = 1$, viés $\beta = 0$, $\epsilon \approx 0$ (para simplificar a conta):

$$\mu = 0 \qquad \sigma^2 = \frac{1}{4}\sum_i (X_i - \mu)^2 = \frac{9+1+1+9}{4} = 5 \qquad \sigma = \sqrt{5} \approx 2{,}236$$

BatchNorm clássica ($\alpha = 1$, sem V_{th}): $z_i = (X_i - \mu)/\sigma$, ou seja $X_i/2{,}236$:

$$z = \{-3/2{,}236,\ -1/2{,}236,\ 1/2{,}236,\ 3/2{,}236\} \approx \{-1{,}34,\ -0{,}45,\ 0{,}45,\ 1{,}34\}$$

tdBN ($V_{th} = 2{,}0$): $Y_i = \alpha V_{th} \cdot z_i = 2 \cdot z_i$ — o mesmo z de cima, só esticado por V_{th}:

$$Y \approx \{-2{,}68,\ -0{,}89,\ 0{,}89,\ 2{,}68\}$$

	BatchNorm clássica ($\sigma = 1$)	**tdBN ($V_{th} = 2{,}0$, $\alpha = 1$)**
Valores normalizados	$[-1{,}34;\ -0{,}45;\ 0{,}45;\ 1{,}34]$	$[-2{,}68;\ -0{,}89;\ 0{,}89;\ 2{,}68]$
Acima de $V_{th} = 2$	**0 de 4** (camada calada)	**1 de 4** (fração ativa)

Normalização dependente de limiar (tdBN): qual a sua importância?

A chave para treinar SNNs profundas diretamente com BPTT

Viabilização de SNNs muito profundas:

Antes da tdBN, o treino com gradiente direto colapsava após 3 a 5 camadas por desvanecimento ou saturação de pulsos.

A tdBN garantiu o primeiro treino supervisionado direto de arquiteturas profundas (ResNet-18, 34 e 50) em dados neuromórficos (ZHENG et al., 2021).

Desacoplamento do limiar de disparo:

A fração de neurônios que ultrapassa o limiar deixa de ser refém do valor numérico arbitrário de V_{th}.

Ajustar parâmetros de membrana (τ, V_{th}) não quebra a escala da camada.

Estabilidade retrógrada de gradientes:

Neurônios sem pulsos geram gradiente substituto nulo. A tdBN assegura atividade contínua, mantendo o sinal de erro vivo até as primeiras camadas.

Referências: (ZHENG et al., 2021; ESHRAGHIAN et al., 2023).

Demonstração ao vivo

SNN → Normalização dependente de limiar (tdBN)

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.tdbn
```

BatchNorm comum fixa variância 1 e, com V_th = 2, cala a camada; tdBN espalha a corrente em $\alpha \cdot V_{th}$, e a fração acima do limiar deixa de depender de V_th.

Regularização de taxa de disparo: o que é?

Mantendo a atividade dos neurônios dentro de uma faixa operacional saudável

Durante a otimização, uma SNN pode aprender adotando comportamentos degenerados. A **regularização de taxa de disparo** insere um custo L_{reg} na função de perda quando a taxa média ρ sai de uma faixa saudável $[r_{\min}, r_{\max}]$:

$$L_{\text{reg}} = \lambda \left[\max(0, r_{\min} - \rho)^2 + \max(0, \rho - r_{\max})^2 \right]$$

onde $\lambda > 0$ é o coeficiente de ponderação, $\rho = \frac{1}{N \cdot T} \sum_{i,t} s_{i,t}$ é a taxa média de disparos da camada (com N neurônios e T passos) e $r_{\min}, r_{\max}$ são os limites da faixa alvo. A penalidade atua contra dois extremos:

Comportamento	**Taxa** ρ	**Consequência**
Silenciados	$\rho < r_{\min}$	Sem aprendizado (gradiente nulo)
Rajadas	$\rho > r_{\max}$	Satura chips e anula economia
Faixa alvo	$[r_{\min}, r_{\max}]$	**Custo zero**: esparsidade com gradientes vivos

Exemplo com $r_{\min} = 0{,}05$, $r_{\max} = 0{,}20$, $\lambda = 10$:

Silenciado ($\rho = 0{,}01$):
$\max(0,\ 0{,}05 - 0{,}01)^2 = 0{,}04^2 = 0{,}0016 \Rightarrow L_{\text{reg}} = 10 \times 0{,}0016 = 0{,}016$.

Faixa alvo ($\rho = 0{,}12$): ambos os termos $\max(0, \text{negativo}) = 0 \Rightarrow L_{\text{reg}} = 0$.

Rajada ($\rho = 0{,}35$):
$\max(0,\ 0{,}35 - 0{,}20)^2 = 0{,}15^2 = 0{,}0225 \Rightarrow L_{\text{reg}} = 10 \times 0{,}0225 = 0{,}225$.

Regularização de taxa de disparo: qual a sua importância?

Garantindo esparsidade neuromórfica e estabilidade de treinamento

Eficiência energética neuromórfica:

Chips neuromórficos (Loihi, SpiNNaker) só gastam energia quando há pulsos (acumulações esparsas).

Disparar em 80% dos passos elimina o benefício energético sobre ANNs. Manter $\rho \in [0{,}05, 0{,}20]$ garante a economia real prometida.

Fluxo contínuo de gradientes:

Gradientes substitutos atuam em uma faixa estreita em torno de V_{th}. Taxas reguladas mantêm os neurônios operando nessa zona ativa de transição.

A sutileza: camada vs. neurônio individual:

Se duas unidades têm taxas individuais $\rho_1 = 0{,}0$ (morta) e $\rho_2 = 1{,}0$ (saturada), a média da camada é $\rho = 0{,}5$ (penalidade zero!).

Técnicas avançadas regularizam cada neurônio individualmente (YAN et al., 2022; ESHRAGHIAN et al., 2023).

Referências: (YAN et al., 2022; ESHRAGHIAN et al., 2023).

Demonstração ao vivo

SNN → Regularização de taxa de disparo

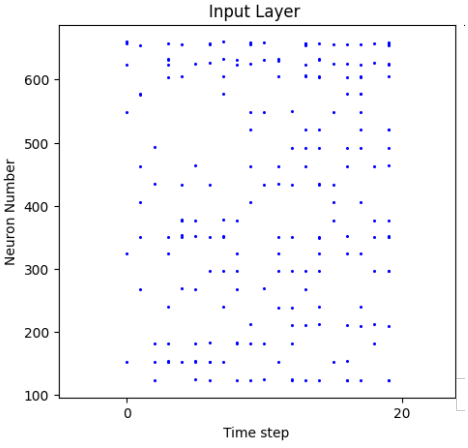
▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh snn.firing_rate_reg
```

Um neurônio quase morto e um em rajada são empurrados de volta à faixa alvo de taxa de disparo; dentro da faixa, a penalidade não faz nada.

Esparsidade: onde mora a eficiência



Eixo horizontal: tempo. Eixo vertical: índice do neurônio. Fonte: elaborado pelo autor.

Na maior parte do tempo, a **grande maioria** dos neurônios está silenciosa.

Pulsos são representados como **uns** dispersamente distribuídos em uma sequência de **zeros** (ESHRAGHIAN et al., 2023).

Um neurônio LIF bem parametrizado tolera ruído: age como um **filtro passa-baixa**, só disparando quando o sinal se acumula de forma sustentada.

Esparsidade: onde mora a eficiência

A promessa, com precisão

Esparsidade só vira economia de energia real em hardware que sabe **pular** o zero. Em GPU/TPU convencional, a esparsidade é invisível ao hardware — ele computa a multiplicação por zero do mesmo jeito.

Volta-se a este ponto na próxima seção, sobre hardware neuromórfico.

Hardware neuromórfico

TrueNorth (IBM, 2014): 1 milhão de neurônios, chip totalmente assíncrono e orientado a evento (MEROLLA et al., 2014).

Loihi (Intel, 2018): aprendizado *on-chip*, arquitetura *manycore* orientada a evento (DAVIES et al., 2018).

SpiNNaker (Manchester, 2014): milhões de núcleos ARM emulando pulsos em tempo real biológico (FURBER et al., 2014).

O modo de falha silencioso

O ganho energético de uma RNP só se realiza **nesse hardware especializado**. Rodada em GPU/TPU convencional — otimizada para multiplicação densa, não para pular zeros — simular T passos de tempo por amostra pode deixar a RNP **mais lenta** que a RNA equivalente.

Aplicações e redes de pulso + *transformers*

Câmeras de eventos (DVS): sensores que já emitem pulsos nativamente — casamento direto com RNPs.

EEG/BCI: sinais biológicos já são fluxos temporais esparsos por natureza — domínio onde a codificação temporal da RNP é vantagem, não custo extra. Para o formalismo de modelos de pulso em neurociência, ver (GOODMAN et al., 2022).

Computação de borda: dispositivos vestíveis e embarcados, onde bateria é o recurso mais escasso.

Spikformer (ZHOU et al., 2023): reformula o mecanismo de atenção do *transformer* em operações compatíveis com pulsos, aproximando as duas linhagens de arquitetura que dominam *deep learning* hoje.

Panorama mais amplo do campo em (ROY; JAISWAL; PANDA, 2019).

Demonstração ao vivo

Comparação → Autoencoders (SNN x LSTM x GRU x Transformer)

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh comparison.autoencoders
```

Como o Meeting01 compara quatro famílias sem trapacear: mesmo gargalo, mesmo alvo, referências média e PCA. A PCA é real (janelas sintéticas); as famílias ficam sem números inventados – só o experimento LOSO os preenche.

Lógica paraconsistente

Por que não só treinar e comparar?

O problema: escolher entre muitos extratores de características

A tese compara extração manual (*wavelet*) com extração automatizada (*autoencoder*) — 24 variantes de *autoencoder*, mais as manuais, para dois sinais. Treinar um classificador para **cada** combinação só para descobrir qual representação é melhor é caro, e pior: o resultado fica amarrado a **qual** classificador se escolheu treinar.

A alternativa: medir a qualidade do vetor de características **antes** de treinar qualquer coisa, olhando só para sua geometria — o quanto cada classe se agrupa (compacidade intraclasse, α) e o quanto classes diferentes se misturam (sobreposição interclasse, β).

O plano paraconsistente

α e β definem um ponto num plano com quatro cantos: **Verdade** (classes compactas e separadas — o alvo), **Falsidade**, **Ambiguidade** e **Indefinição**. Quanto mais perto de Verdade, melhor a representação separa as classes **sem treinar classificador nenhum**.

As próximas demos mostram esse plano, uma falha dele que passou despercebida até ser medida, e a correção — a contribuição central desta tese.

Demonstração ao vivo

Paraconsistente → Plano paraconsistente

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh paraconsistent.plane
```

Alpha (compacidade intraclasse) e beta (sobreposição interclasse) definem um ponto no plano, com quatro cantos: Verdade, Falsidade, Ambiguidade, Indefinição.

Demonstração ao vivo

Paraconsistente → D_truth x D_penalized

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh paraconsistent.dpenalized
```

Por D_truth, um extrator ”morto”($\sqrt{2} \approx 1{,}41$) passa à frente de um fraco porém real (1,50); D_penalized leva os três cantos ruins a exatamente 2 e fecha essa brecha – contribuição central da tese do autor.

Demonstração ao vivo

Paraconsistente → Busca genética de arquiteturas (extensão)

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh paraconsistent.ga_pareto
```

Extensão fora da monografia: o software/nn usa D_penalized numa busca NSGA-II. A fronteira de Pareto reúne arquiteturas em que nenhuma é melhor em tudo; a regra de Deb descarta as que excedem o teto de latência.

Desafios e síntese

Limitações do experimento

A limitação dominante: tamanho da base

15 locutores, 1974 amostras. Sob protocolo de verificação — partições disjuntas por locutor — cada dobra treina com poucas identidades: risco alto de o classificador não ter sinal suficiente para generalizar.

As três categorias de representação (manual, *autoencoder*, fusão) **não** foram comparadas por um classificador comum — só pelo critério paraconsistente.

Os *autoencoders* tiveram $\bar{\beta} > 0{,}97$ em toda configuração: latentes perto do colapso. Não dá para distinguir, com os dados disponíveis, “extrator pior” de “treino insuficiente”.

Consequência

A seleção paraconsistente ranqueia qualidade **antes** de treinar um classificador — mas ranquear bem um extrator degenerado via treino insuficiente ainda é um risco que $D_{\text{penalizado}}$ por si só não resolve.

Síntese

Extração manual venceu a seleção paraconsistente para voz e EEG, com a mesma configuração (*wavelet* Haar, LFCC) — nenhuma das 24 variantes de *autoencoder* se aproximou.

Autenticação ficou perto do acaso: melhor EER $= 0{,}4534$, melhor AUC $= 0{,}5452$, contra 0,5/0,5 de um classificador aleatório — um resultado negativo, não um ajuste fino pendente.

EEG e fusões superaram voz isolada entre as configurações mais bem colocadas — consistente com a motivação de complementar fala degradada com sinal cerebral.

Três contribuições metodológicas

$D_{\text{penalizado}}$ — fecha a brecha dos três cantos degenerados do plano paraconsistente.

Diagnóstico do piso de ruído estrutural de codificações temporais (`direct` $\succ$ `latency` $\succ$ `poisson`).

Protocolo em duas fases: seleção do extrator separada da medição de desempenho.

Demonstração ao vivo

SNN → Classificação de vogais ao vivo (microfone)

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh live.vowel_snn
```

Ligar o microfone, falar uma vogal (a/e/i/o/u) e mostrar os disparos das camadas oculta e de saída, com a confiança por vogal ao vivo.

Demonstração ao vivo

SNN → Classificação de vogais ao vivo (neurônios)

▶ Abrir no software

ou, no terminal (a partir desta pasta):

```
./abrir-demo.sh live.vowel_snn_nodes
```

Mesmo classificador ao vivo, agora mostrando os próprios neurônios – entrada, oculta e saída – e os pesos aprendidos (W1, W2) entre eles, em vez do gráfico de disparos no tempo.

Obrigado.

André Furlan — ensismoebius@gmail.com

Referências I

DAVIES, M. et al. Loihi: A neuromorphic manycore processor with on-chip learning. *IEEE Micro*, v. 38, n. 1, p. 82–99, 2018.

DIEHL, P. U. et al. Fast-classifying, high-accuracy spiking deep networks through weight and threshold balancing. In: *2015 International Joint Conference on Neural Networks (IJCNN)*. [S.l.]: IEEE, 2015. p. 1–8.

ESHRAGHIAN, J. K. et al. Training spiking neural networks using lessons from deep learning. *Proceedings of the IEEE*, v. 111, n. 9, p. 1016–1054, 2023.

FURBER, S. B. et al. The spinnaker project. *Proceedings of the IEEE*, v. 102, n. 5, p. 652–665, 2014.

Referências II

GERSTNER, W. et al. *Neuronal Dynamics: From Single Neurons to Networks and Models of Cognition*. Cambridge University Press, 2014. ISBN 9781107060838. Disponível em: ⟨https://neuronaldynamics.epfl.ch/online/Ch2.S2.html⟩.

GOODMAN, D. et al. *Spiking Neural Network Models in Neuroscience - Cosyne Tutorial 2022*. Zenodo, 2022. Disponível em: ⟨https://doi.org/10.5281/zenodo.7044500⟩.

KASABOV, N. K. *Time-space, spiking neural networks and brain-inspired artificial intelligence*. [S.l.]: Springer, 2019.

LIU, S.; LEUNG, V. C. H.; DRAGOTTI, P. L. First-spike coding promotes accurate and efficient spiking neural networks for discrete events with rich temporal structures. *Frontiers in Neuroscience*, v. 17, p. 1266003, 2023.

Referências III

MAASS, W. Networks of spiking neurons: The third generation of neural network models. *Neural Networks*, v. 10, n. 9, p. 1659–1671, 1997.

MEROLLA, P. A. et al. A million spiking-neuron integrated circuit with a scalable communication network and interface. *Science*, v. 345, n. 6197, p. 668–673, 2014.

ROY, K.; JAISWAL, A.; PANDA, P. Towards spike-based machine intelligence with neuromorphic computing. *Nature*, v. 575, n. 7784, p. 607–617, 2019.

RUECKAUER, B. et al. Conversion of continuous-valued deep networks to efficient event-driven networks for image classification. *Frontiers in Neuroscience*, v. 11, p. 682, 2017.

Referências IV

WU, Y. et al. Spatio-temporal backpropagation for training high-performance spiking neural networks. *Frontiers in Neuroscience*, v. 12, p. 331, 2018.

YAN, Y. et al. Backpropagation with sparsity regularization for spiking neural network learning. *Frontiers in Neuroscience*, v. 16, p. 760298, 2022.

ZHENG, H. et al. Going deeper with directly-trained larger spiking neural networks. In: *Proceedings of the AAAI Conference on Artificial Intelligence*. [S.l.: s.n.], 2021. v. 35, n. 12, p. 11062–11070.

ZHOU, Z. et al. Spikformer: When spiking neural network meets transformer. In: *International Conference on Learning Representations (ICLR)*. [S.l.: s.n.], 2023.